Virginia Market Square Website Redesign - Advanced Capstone Proposal

Full-Stack E-Commerce Platform with Vendor Self-Service & Advanced Database

Prepared by: Web Development Capstone Team
Date: February 2026
Project Duration: 6 weeks (March 16 - May 1, 2026)
Estimated Total Hours: 52 hours
Target Skill Level: Strong HTML/CSS, Bootstrap proficiency, intermediate PHP, introductory SQL

Executive Summary

This advanced capstone project proposes a comprehensive website redesign for **Virginia Market Square**, a non-profit farmers market in Virginia, Minnesota. Rather than a simplified version, this proposal includes sophisticated features reflecting professional e-commerce standards: vendor self-service logins with profile management, customer shopping carts with database persistence, multi-step checkout processes, payment gateway integration with Stripe, and a normalized relational database with advanced SQL queries using JOINs.

The project represents a significant learning opportunity, moving from basic web development to full-stack e-commerce architecture. The developer will design and implement a proper relational database with normalization, learn authentication systems for multiple user roles, understand complex SQL queries with table relationships, and integrate third-party payment processing. Within 52 hours over six weeks, this project will deliver a production-quality website that genuinely serves the organization while providing portfolio-quality evidence of intermediate-to-advanced web development capabilities.

The scope is ambitious but realistic, with careful prioritization ensuring core features launch by May 1 while optional enhancements are identified for Phase 2 implementation.

Project Vision & Expanded Scope

Project Name

Virginia Market Square: Full-Stack E-Commerce Local Market Platform

Vision Statement

Create a professional, feature-rich website that empowers local vendors to showcase and sell their products directly to consumers, while providing customers with a seamless shopping experience and administrators with comprehensive management tools—all built on a properly normalized relational database with secure authentication for three distinct user types.

What Makes This Advanced

The original simplified proposal included basic vendor listings and static content management. This advanced version includes:

For Customers:

- Browse and search products across all vendors
- Create accounts and manage profiles
- Add items to shopping cart (database-backed, persistent)
- Complete multi-step checkout process
- Track order status and history
- Secure payment processing via Stripe

For Vendors:

- Self-service account creation and login
- Complete profile management
- Add, edit, and delete products independently
- View orders containing their products
- Track sales metrics and revenue
- Analytics dashboard with revenue calculations

For Administrators:

- Approve vendor applications and create accounts
- Manage all vendor, product, and order data
- View site-wide analytics and sales reports
- Manage events and content

Technology Foundation:

- Properly normalized relational database (12 tables, 3NF)
- Foreign key relationships between all tables

- Complex SQL queries using JOINs for meaningful data retrieval
- Secure authentication system with role-based access control
- Three distinct user types with permission levels
- Payment gateway integration (Stripe)
- Session-based authorization with CSRF protection

Current Website Analysis & Project Justification

The existing Squarespace-based website suffers from significant limitations preventing the organization from fully serving its mission. The platform cannot support vendor self-service management, lacks any e-commerce capability, provides no means for customers to discover products beyond basic vendor listings, and cannot track sales or revenue. More fundamentally, Squarespace's proprietary architecture prevents deep customization and creates vendor lock-in, with annual costs exceeding $200 for premium features the organization needs.

This advanced capstone project will deliver a custom-built platform that not only addresses these limitations but provides a solid foundation for long-term growth. The organization will own its complete web presence, control all functionality, and have the technical foundation to add features independently or with future developers. More importantly for this capstone, the project provides comprehensive learning in full-stack web development, database design, authentication systems, and payment integration—skills that directly map to professional development positions.

Advanced Database Architecture

Normalized Relational Design

Rather than a simplified flat structure, the database uses Third Normal Form (3NF) normalization principles, eliminating data duplication and enabling complex queries. The schema includes 12 primary tables with foreign key relationships creating a robust data model supporting multiple features simultaneously.

Core Tables (Database Design)

Users Table: Central authentication repository for all three user types (admin, vendor, customer) with hashed passwords and role designation.

Vendors Table: Links to users via foreign key, stores vendor business information including verification status, location validation, and profile content.

Products Table: Product inventory with normalized category references, stock tracking, pricing, and availability status. Includes foreign key to vendors (each product belongs to one vendor).

Categories Table: Separate table for product categories, enabling consistent categorization without duplicating category names across products.

Customers Table: Customer profile information for e-commerce transactions, linked to users table, storing shipping addresses and contact information.

Orders Table: Complete order records with status and payment tracking, linked to customers via foreign key.

Order_Items Table: Individual line items in orders, creating a many-to-many relationship between orders and products (one order can contain many products from potentially different vendors).

Cart Table: Persistent shopping cart stored in database, enabling customers' carts to survive browser refresh or multi-session shopping.

Transactions Table: Payment transaction records for audit trails and reconciliation, linked to orders via foreign key.

Events Table: Market dates, special events, and seasonal information.

Contacts Table: Contact form submissions, optionally linked to specific vendors.

Vendor_Applications Table: New vendor applications awaiting admin approval.

Foreign Key Relationships

The database implements referential integrity through foreign keys. Every vendor record references a user record; every product references a vendor; every order references a customer; every order item references both product and vendor. This ensures data consistency—a product cannot exist without a vendor, an order cannot exist without a customer, transactions cannot exist without orders.

Advanced Queries Using JOINs

The normalized structure enables powerful queries combining data across multiple tables. A single query can retrieve a customer's order, all items in that order, the vendor information for each product, and pricing history—all in one result set. This capability powers the vendor

dashboard (calculating revenue by joining orders, order_items, and products tables), the shopping cart display (joining cart items with product and vendor information), and customer order tracking.

Multi-User Authentication System

Three User Roles with Distinct Permissions

Customer: Can browse products, create accounts, manage shopping cart, complete purchases, view order history.

Vendor: Can manage own profile and products, view orders containing their products, track sales and revenue, but cannot access admin functions or other vendors' data.

Admin: Full access to all site functions, can manage vendors, products, orders, users, and content.

Secure Authentication Implementation

Each user type uses the same users table with a user_type enum field determining permissions. Passwords are hashed using PHP's password_hash() function with BCRYPT, making them irreversible and secure even if the database is compromised. Sessions use PHP's built-in session management with proper timeout and regeneration on login.

Authorization is enforced at the page level—every vendor page checks that the logged-in user is a vendor and can only access their own data. A vendor cannot view another vendor's products or orders; a customer cannot access admin functions. This role-based access control (RBAC) pattern is fundamental to professional web applications.

Vendor Registration & Approval Workflow

Prospective vendors submit applications through the website, with data stored in the vendor_applications table. Administrators review applications in the admin dashboard and either approve or reject them. Upon approval, the administrator creates a user account (with temporary password), which creates a corresponding vendor record in the vendors table. The vendor then logs in with their credentials and can immediately begin managing products.

E-Commerce System Architecture

Shopping Cart with Database Persistence

Unlike simple session-based shopping carts that disappear on browser close, the cart is stored in the database's cart table. When a customer adds a product, a record is inserted into the cart table linked to their customer ID. When they view their cart, a query retrieves all cart items joined with product and vendor information. This approach has several advantages: carts persist across sessions, calculations are done server-side (more secure), and administrators can see abandoned carts or inventory demand.

Multi-Step Checkout Process

Checkout is divided into five distinct steps, each validating and persisting data:

Step 1: Review cart and confirm items
Step 2: Enter or select shipping address
Step 3: Choose shipping method (pickup vs. delivery)
Step 4: Confirm order and review totals
Step 5: Payment processing via Stripe

This progressive disclosure prevents overwhelming customers while enabling data validation at each stage. Session data maintains the customer's checkout progress if they navigate backward.

Order Creation from Cart

When checkout completes successfully, an atomic transaction converts the shopping cart into an order. A single order record is created with status 'pending' and payment_status 'unpaid'. For each item in the cart, an order_item record is created capturing the product ID, quantity, unit price (at time of purchase), and the vendor ID. The cart is then cleared for that customer.

This design ensures accurate order history even if product prices change afterward—the order_item records preserve historical pricing. The design also enables vendor-specific order views by joining order_items with the vendor ID.

Stripe Payment Integration

Payment processing integrates with Stripe, a professional payment gateway handling credit card processing securely. Rather than the website storing credit card data (which requires PCI compliance and significant security infrastructure), Stripe tokenizes the card information and the website receives only a token. The server-side code uses this token to create a charge via Stripe's API.

If payment succeeds, the order_id and transaction details are recorded in the transactions table. If payment fails, an appropriate error message is displayed and the checkout form

remains populated for the customer to retry. This integration provides real-world experience with API integration and payment processing workflows.

Vendor Sales Tracking

Vendors view orders through a dashboard query that joins orders, order_items, products, and vendors tables. A vendor sees only orders containing their products, with calculations showing their share of revenue. SQL aggregate functions (SUM, COUNT) calculate total revenue, item count, and per-order totals. DATE functions enable viewing sales by day, week, or month. This demonstrates sophisticated SQL querying combining normalization benefits with business intelligence.

Frontend Architecture

Responsive Bootstrap Design

The user interface uses Bootstrap 5 framework with customization for the farmers market brand. All pages are mobile-first responsive, adapting to screens from 375 pixels (mobile) to 1920+ pixels (large desktop). Bootstrap's grid system, pre-built components (navbars, cards, forms, tables), and utility classes provide a professional foundation.

Custom CSS overlays Bootstrap's defaults with the organization's color scheme (greens and earth tones), typography choices, and specific component styling. The result is a cohesive, professional interface that appears custom-built while leveraging Bootstrap's foundation and component library.

Key Frontend Pages (25+ Total)

Public Pages (Customers & Guests):

- Homepage with featured products and market information
- Product browsing with category filtering and search
- Vendor storefronts showing all products from one vendor
- Individual product detail pages
- Shopping cart management
- Five-step checkout process
- Order confirmation and tracking
- Customer login and registration
- Events calendar
- Contact form
- Information pages (About, Mission, Vendor Requirements)

Vendor Pages (Self-Service):

- Vendor login and dashboard
- Vendor profile management
- Product management (add/edit/delete)
- Sales analytics and revenue tracking
- Orders containing vendor's products
- Monthly/weekly/daily sales reports

Admin Pages (Management):

- Admin login and dashboard
- Vendor application review and approval
- All vendor management
- All order management
- Site-wide analytics
- Content and event management
- User account management

User Experience Details

Each page is designed around the user's current task. The product browsing page emphasizes discovery through filtering and search. The shopping cart page focuses on review and modification. The checkout pages guide the customer through required steps with clear progress indication. The vendor dashboard prioritizes sales metrics and recent orders. This task-focused design makes functionality discoverable and reduces friction.

Advanced SQL Query Examples

Example 1: Customer Shopping Cart Display

```
SELECT
    c.cart_id,
    c.quantity,
    p.product_id,
    p.product_name,
    p.price,
    v.business_name,
    v.vendor_id,
    (c.quantity * p.price) AS line_total
FROM cart c
INNER JOIN products p ON c.product_id = p.product_id
```

```
INNER JOIN vendors v ON p.vendor_id = v.vendor_id
WHERE c.customer_id = 10
ORDER BY v.business_name, p.product_name;
```

This query retrieves all cart items with product and vendor information in a single query, enabling the shopping cart page to display all necessary details without multiple sequential queries.

Example 2: Vendor Sales Dashboard

```
SELECT
    DATE_FORMAT(o.order_date, '%Y-%m-%d') AS date,
    v.business_name,
    COUNT(DISTINCT o.order_id) AS order_count,
    SUM(oi.quantity) AS items_sold,
    SUM(oi.line_total) AS daily_revenue
FROM vendors v
INNER JOIN products p ON v.vendor_id = p.vendor_id
INNER JOIN order_items oi ON p.product_id = oi.product_id
INNER JOIN orders o ON oi.order_id = o.order_id
WHERE v.vendor_id = 5 AND o.status = 'confirmed'
GROUP BY DATE_FORMAT(o.order_date, '%Y-%m-%d')
ORDER BY date DESC
LIMIT 30;
```

This query calculates vendor revenue by date using joins across four tables and date formatting functions, demonstrating SQL's analytical capabilities.

Example 3: Top Products by Revenue

```
SELECT
    p.product_id,
    p.product_name,
    v.business_name,
    COUNT(DISTINCT oi.order_id) AS times_ordered,
    SUM(oi.quantity) AS total_units_sold,
    SUM(oi.line_total) AS total_revenue,
    AVG(oi.unit_price) AS avg_price
FROM products p
INNER JOIN vendors v ON p.vendor_id = v.vendor_id
INNER JOIN order_items oi ON p.product_id = oi.product_id
```

```
WHERE oi.vendor_id = 5
GROUP BY p.product_id
ORDER BY total_revenue DESC
LIMIT 20;
```

This demonstrates grouping, aggregation, and filtering to identify top-performing products for a vendor.

Skills Development Through This Project

Database Design & Normalization

Students learn to design databases following Third Normal Form principles, eliminating data duplication and ensuring data integrity. They understand the balance between normalization (preventing redundancy) and denormalization (query performance), making informed decisions about table structure.

Advanced SQL & Table Joins

Moving from simple SELECT queries, students write complex queries combining data from 4-5 tables simultaneously. They understand INNER JOIN (returns only matching rows), LEFT JOIN (includes non-matching rows from left table), and when each is appropriate. They use WHERE clauses to filter complex result sets and ORDER BY to sort properly. They learn aggregate functions (SUM, COUNT, AVG) for analytical queries.

Authentication & Authorization

Students implement real security practices: password hashing with BCRYPT, session-based authentication, role-based access control checking user permissions before displaying data, CSRF token protection against cross-site request forgery attacks. They understand security vulnerabilities (SQL injection, XSS) and how to prevent them.

Payment Gateway Integration

Working with Stripe provides exposure to third-party API integration, understanding how production websites handle sensitive payment data securely, and implementing proper error handling for transaction failures. This is directly applicable to professional development roles.

Full-Stack Architecture

Students see how all pieces fit together: database storing data, PHP retrieving and manipulating data, HTML/CSS/JavaScript presenting it, and external services (Stripe) integrating into the system. This systems thinking is essential for professional development.

Project Deliverables

Technical Deliverables

- Fully functional website with all features live on production hosting
- Complete relational database with 12 tables, proper normalization, and indexes
- 25+ PHP pages covering customer, vendor, and admin functions
- Advanced SQL queries demonstrating JOINs, aggregation, and analytics
- Stripe payment integration (test mode functional)
- Complete source code repository on GitHub
- Database schema documentation with ER diagram
- API documentation for custom functions

Documentation

- Comprehensive README with setup and installation instructions
- Database schema guide with table relationships explanation
- Admin user manual with screenshots
- Vendor user manual explaining dashboard and product management
- Customer guide for shopping and checkout
- Security audit documentation
- Deployment guide for future hosting changes

Demonstration Materials

- Sample data populated in database (realistic vendors, products, orders)
- Screenshots of key features
- Video walkthrough of main workflows
- Performance metrics and optimization notes

Task Breakdown & Timeline

Phase	Tasks	Estimated Hours
1: Planning & Database Design	• Create a normalized 12-table schema with an ER diagram and foreign keys. • Specify requirements for three user types. • Outline user flows and wireframes . • Prepare visual mockups, branding, and Bootstrap components. • Research Stripe API integration and detail the payment workflow and necessary libraries.	6 hrs
2: Database Setup & Authentication	• 12 indexed database tables with realistic data. • role-based authentication • session management for admin vendor, and customer accounts • shared authentication and permission utilities	7 hrs
3: Product Browsing & Backend	• Create a product catalog with search, filters, detail pages, and vendor storefronts using JOINs • add category browsing, vendor profiles, and feature products on the homepage • rebuild supporting pages with database integration	8 hrs
4: Shopping Cart & Checkout	• Build a database cart with item controls • implement a five-step checkout with validation and session tracking • convert carts to orders • add order history and tracking pages	8 hrs
5: Vendor Dashboard & Analytics	• Create a dashboard with SQL-based sales and revenue metrics • product management tools • period-specific sales reports • real-time vendor order statuses.	3 hrs
6: Frontend & Responsive Design	• Integrate Bootstrap 5, customize styles for branding; • develop mobile-first responsive layouts across 25+ pages; • ensure intuitive UX, accessible color contrast, clear calls to action	8 hrs

7: Payment & JavaScript	• Integrate Stripe for payments, handle validation and transaction feedback; • enable client-side form validation, dynamic cart updates with JavaScript; • add interactive enhancements like animations and lazy loading	3 hrs
8: Content & Testing	• Populate database with sample vendors, products, events • test all core features • review security measures	2 hrs
9: Deployment & Launch	• Set up hosting, MySQL, domain • test production functionality • finalize documentation • go live May 1, 2026	1 hr

---

Timeline

Week	Phase	Focus	Hours
1 (Mar 9)	1-2	Planning & Database	6
2 (Mar 16)	2-3	Auth & Backend Start	7
3 (Mar 23)	3	Product Pages	8
4 (Apr 30)	4	Cart & Checkout	8
5 (Apr 6)	5-6	Dashboard & Frontend	8
6 (Apr 13)	6-7	Design & Payment	8
7 (Apr 20)	8-9	Testing & Launch	5
TOTAL			**52**

Average: 8-9 hours per week, achievable alongside coursework with focused effort.

Prioritization for 52-Hour Constraint

Must-Have Features (35-40 hours)

- All authentication systems for three user types
- Product browsing with JOINs
- Database-backed shopping cart
- Multi-step checkout process (payments optional)
- Vendor dashboard
- Responsive Bootstrap design
- Deployment and documentation

Should-Have Features (8-12 hours)

- Stripe payment integration
- Vendor analytics dashboard
- Advanced searching and filtering
- Email notifications

Nice-to-Have Features (if time permits)

- Product reviews and ratings
- Wishlist functionality
- Inventory alerts
- Mobile app

If running short on schedule, Priority 1 features will launch. Payment processing can be added post-launch as Phase 2. This ensures a working e-commerce site regardless of timeline pressures.

Conclusion

This advanced capstone project represents a significant but achievable undertaking in full-stack web development. The scope—a functioning e-commerce platform with proper database normalization, multi-user authentication, shopping cart, checkout process, and payment integration—matches professional standards while remaining realistic for 52 hours over six weeks.

The developer will graduate from template-based websites to designing systems, from simple CRUD operations to complex analytical queries, from basic authentication to role-based access control. The skills acquired—database normalization, SQL JOINs, authentication

systems, payment gateway integration—are directly applicable to professional development positions.

The finished product serves Virginia Market Square's actual needs while providing portfolio evidence of serious web development capabilities. Rather than another portfolio project with fake data and limited functionality, this represents a real website solving real problems.

Launch April 27, 2026. Iterate with Phase 2 features as time and resources allow.